

Introduction to Distributed Systems

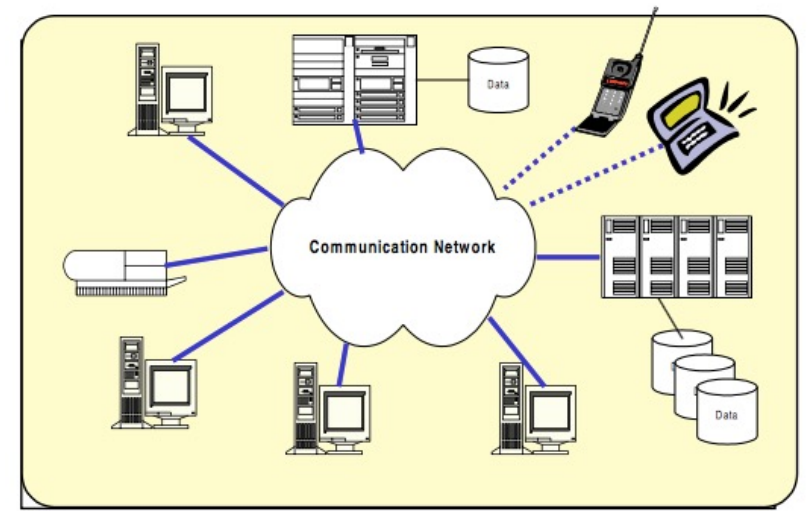
Thoai Nam

High Performance Computing Lab (HPC Lab)
Faculty of Computer Science and Engineering
HCMC University of Technology

Applications

<i>Finance and commerce</i>	eCommerce e.g. Amazon and eBay, PayPal, online banking and trading
<i>The information society</i>	Web information and search engines, ebooks, Wikipedia; social networking: Facebook and MySpace.
<i>Creative industries and entertainment</i>	online gaming, music and film in the home, user-generated content, e.g. YouTube, Flickr
<i>Healthcare</i>	health informatics, on online patient records, monitoring patients
<i>Education</i>	e-learning, virtual learning environments; distance learning
<i>Transport and logistics</i>	GPS in route finding systems, map services: Google Maps, Google Earth
<i>Science</i>	The Grid as an enabling technology for collaboration between scientists
<i>Environmental management</i>	sensor technology to monitor earthquakes, floods or tsunamis

A Distributed System is ...



- Multiple connected CPUs working together
 - Components located at networked computers communicate and coordinate their actions only by message passing
 - A collection of independent computers that appears to its users as a single coherent system.
 - A collection of autonomous computers interconnected by a computer network and equipped with distributed system software to form an integrated computing facility.
 - Multiple independent machines interconnected through a network to coordinate to achieve a common, consistent service or function.
-
- Examples: networked machines, Internet, Intranet, mobile and ubiquitous computing

A Distributed System is ...

- “... a system in which the failure of a computer you didn’t even know existed can render your own computer unusable.” — Leslie Lamport
- ... multiple computers communicating via a network ...
- ... trying to achieve some task together
- Consists of “nodes” (computer, phone, car, robot, ...).

A Distributed System is ...

- “... a system in which the failure of a computer you didn’t even know existed can render your own computer unusable.” — Leslie Lamport
- ... multiple computers communicating via a network ...
- ... trying to achieve some task together
- Consists of “nodes” (computer, phone, car, robot, ...).

Advantages

- It's inherently distributed
 - e.g. sending a message from your mobile phone to your friend's phone
 - Communication and resource sharing possible
- For better reliability
 - even if one node fails, the system as a whole keeps functioning
- For better performance
 - get data from a nearby node rather than one halfway round the world
- Scalability
 - To solve bigger problems
 - e.g. huge amounts of data, can't fit on one machine
- Potential for incremental growth
- Economics – price-performance ratio

Disadvantages

- Distribution-aware PLs, OSs and applications
- Security and privacy
- Network connectivity essential
 - Communication may fail (and we might not even know it has failed).
 - Processes may crash (and we might not know).
 - All of this may happen nondeterministically.

Fault tolerance: we want the system as a whole to continue working, even when some parts are faulty.

This is hard.

Writing a program to run on a single computer is comparatively easy?!

Challenges

- Heterogeneity
- Openness
- Security
- Scalability
- Failure handling
- Concurrency
- Transparency

Transparency

Transparency	Description
<i>Access</i>	Enables local and remote resources to be accessed using identical operations
<i>Location</i>	Enables resources to be accessed without knowledge of their physical or network location (for example, which building or IP address)
<i>Concurrency</i>	Enables several processes to operate concurrently using shared resources without interference between them
<i>Replication</i>	Enables multiple instances of resources to be used to increase reliability and performance without knowledge of the replicas by users or application programmers
<i>Failure</i>	Enables the concealment of faults, allowing users and application programs to complete their tasks despite the failure of hardware or software components
<i>Mobility</i>	Allows the movement of resources and clients within a system without affecting the operation of users or programs
<i>Performance</i>	Allows the system to be reconfigured to improve performance as loads vary
<i>Scaling</i>	Allows the system and applications to expand in scale without change to the system structure or the application algorithms

Scalability problems

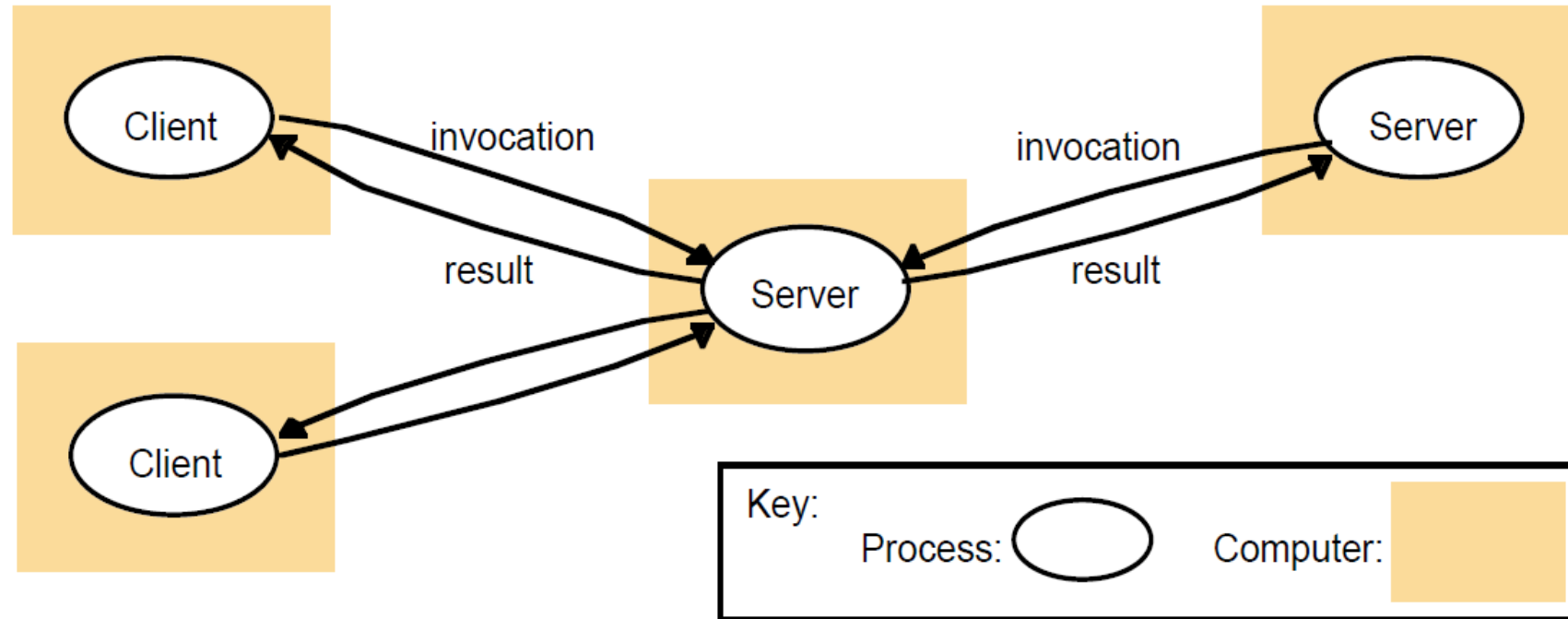
Concept	Example
Centralized services	A single server for all users
Centralized data	A single on-line telephone book
Centralized algorithms	Doing routing based on complete information

Architecture models

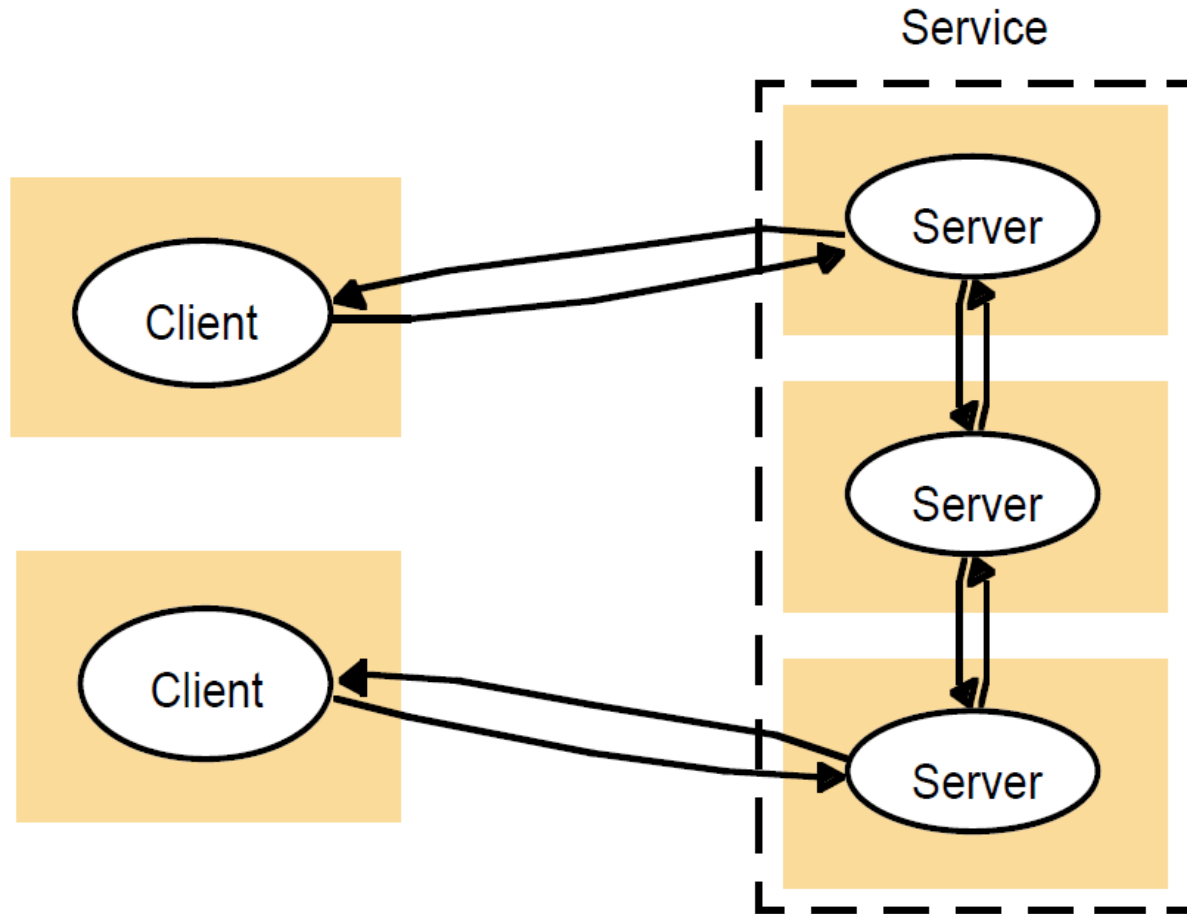
Main types of models

- Client-server
 - first and most commonly used
- Multiple servers
 - to improve performance and reliability
 - e.g. search engines (1000's of computers)
- Proxy servers
 - to reduce load on network, provide access through firewall
- Peer processes
 - when faster interactive response needed.

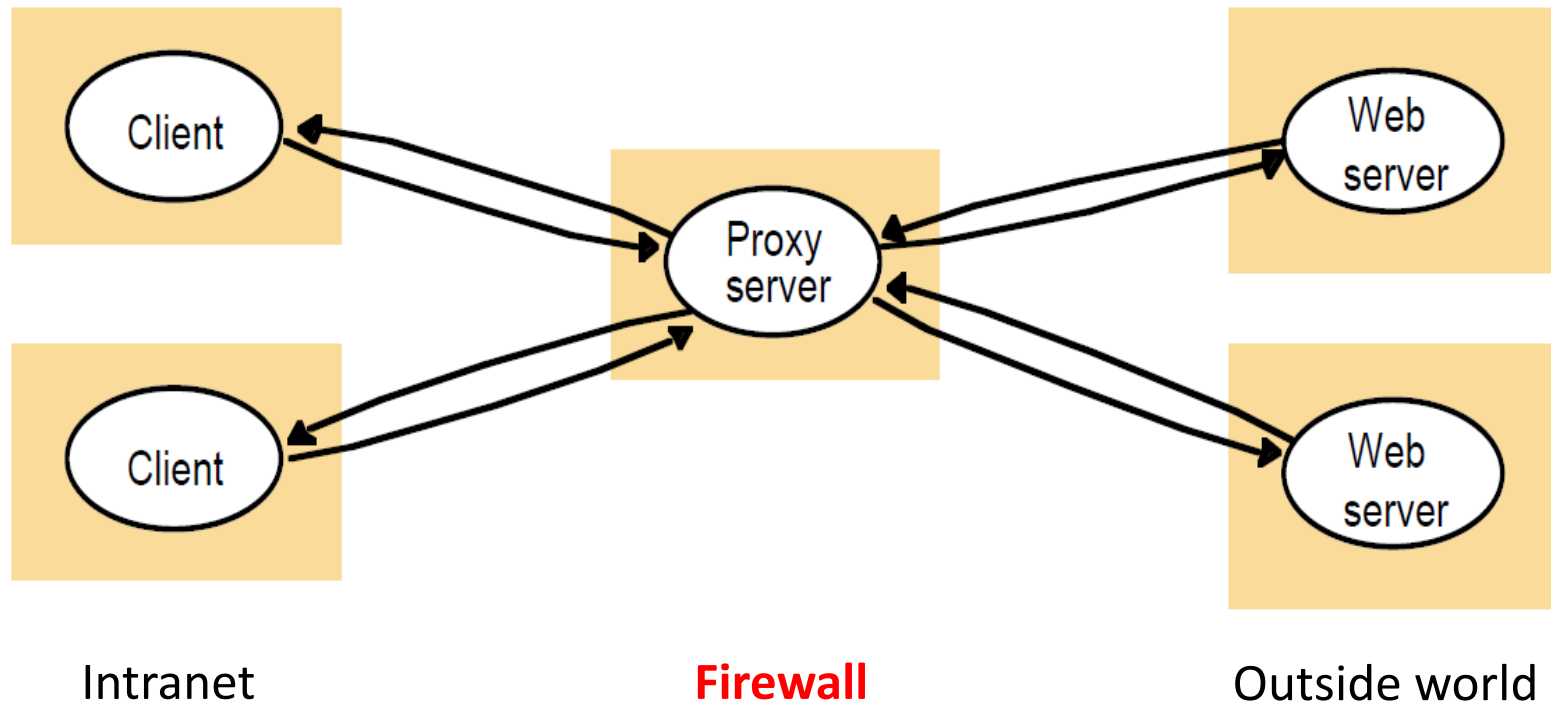
Client server



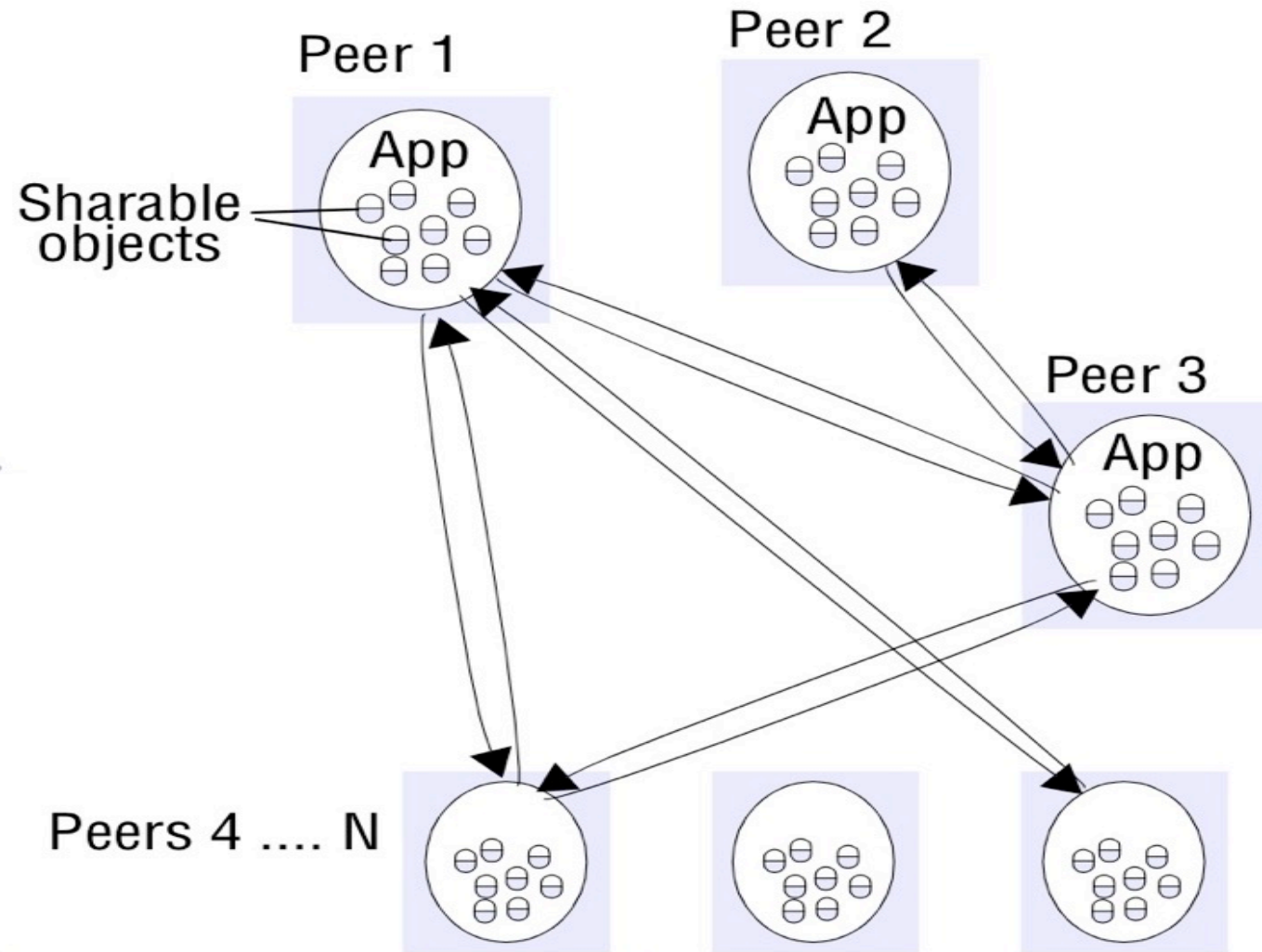
A service provided by multiple servers



Proxy servers



Peer-to-peer



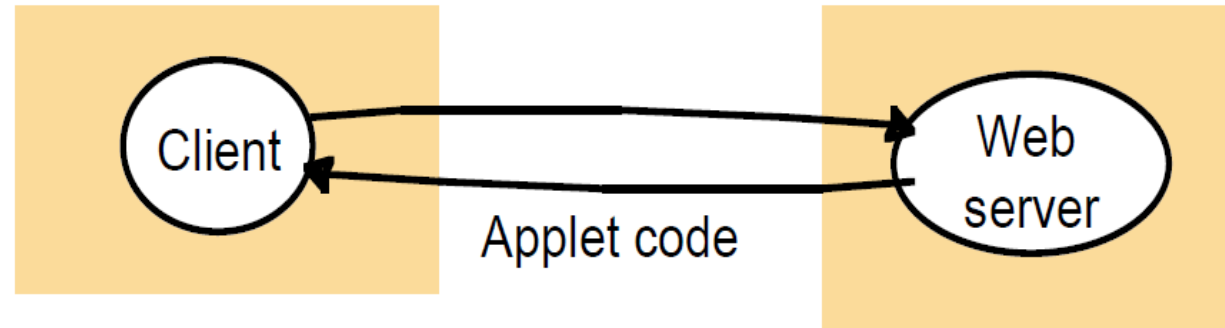
Client server and mobility

- Mobile code
 - downloaded from server, runs on locally
 - e.g. web applets
- Mobile agent (code + data)
 - travels from computer to another
 - collects information, returning to origin

Beware! Security risks

Web applets

a) client request results in the downloading of applet code



b) client interacts with the applet



Design requirements for DSs

Judging how good the architecture is...

- **Performance**
 - how fast will it respond?
- **Quality of Service**
 - are video frames and sound synchronized?
- **Dependability**
 - does it work correctly?

Performance

- **Responsiveness**
 - fast interactive response delayed by remote requests
 - use of caching, replication
- **Throughput**
 - dependent on speed of server and data transfer
- **Load balancing**
 - use of applets, multiple servers

Quality of Service (QoS)

Non-functional properties experienced by users:

- **Deadline properties**
 - **hard** deadlines (must be met within T time units)
 - **soft** deadlines (there is a 90% chance that the video frame will be delivered within T time units)
 - ✧ Multi media traffic, video/sound synchronization
 - ✧ depend on availability of sufficient resources
- **Adaptability**
 - ability to adapt to changing system configuration

Dependability

- **Correctness**
 - Dependability
 - correct behavior wrt specification – e.g. use of verification
- **Fault-tolerance**
 - ability to tolerate/recover from faults – e.g. use of redundancy
- **Security**
 - ability to withstand malicious attack – e.g. use of encryption, etc.

Distributed Operating Systems

Multiprocessor Operating Systems

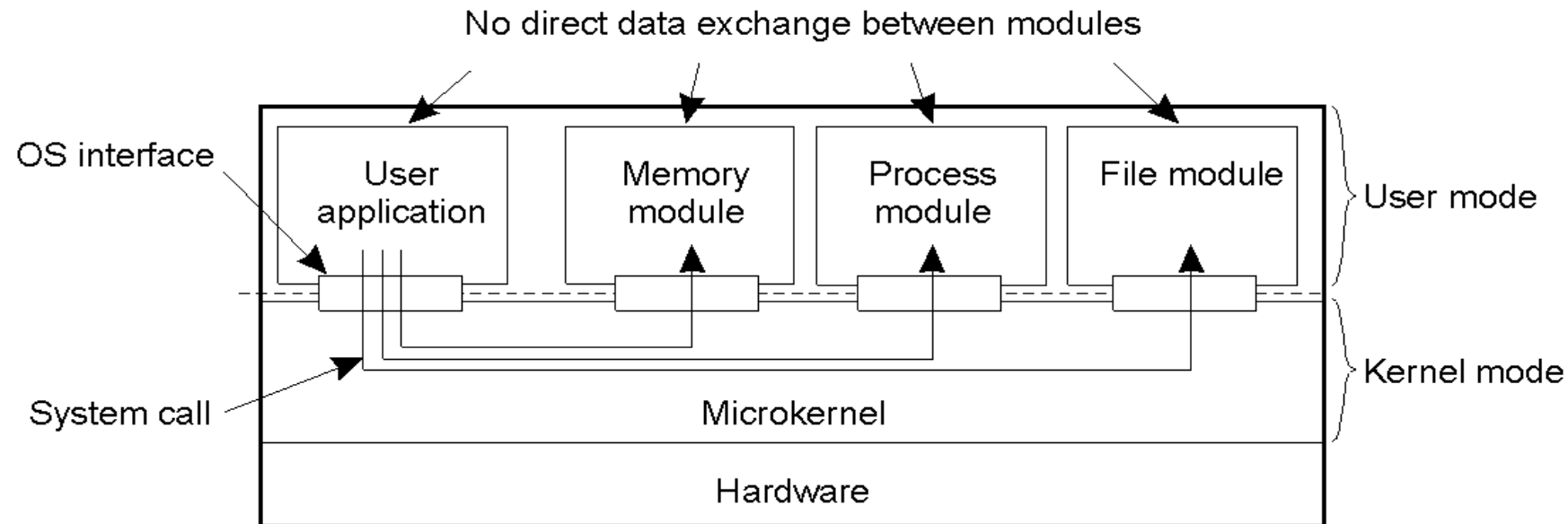
- Like a uniprocessor operating system
- Manages multiple CPUs transparently to the user
- Each processor has its own hardware cache
 - Maintain consistency of cached data

Uniprocessor Operating Systems

- An OS acts as a resource manager or an arbitrator
 - Manages CPU, I/O devices, memory
- OS provides a virtual interface that is easier to use than hardware
- Structure of uniprocessor operating systems
 - Monolithic (e.g., MS-DOS, early UNIX)
 - ✧ One large kernel that handles everything
 - Layered design
 - ✧ Functionality is decomposed into N layers
 - ✧ Each layer uses services of layer N-1 and implements new service(s) for layer N+1

Uniprocessor Operating Systems

- Microkernel architecture
 - Small kernel
 - user-level servers implement additional functionality



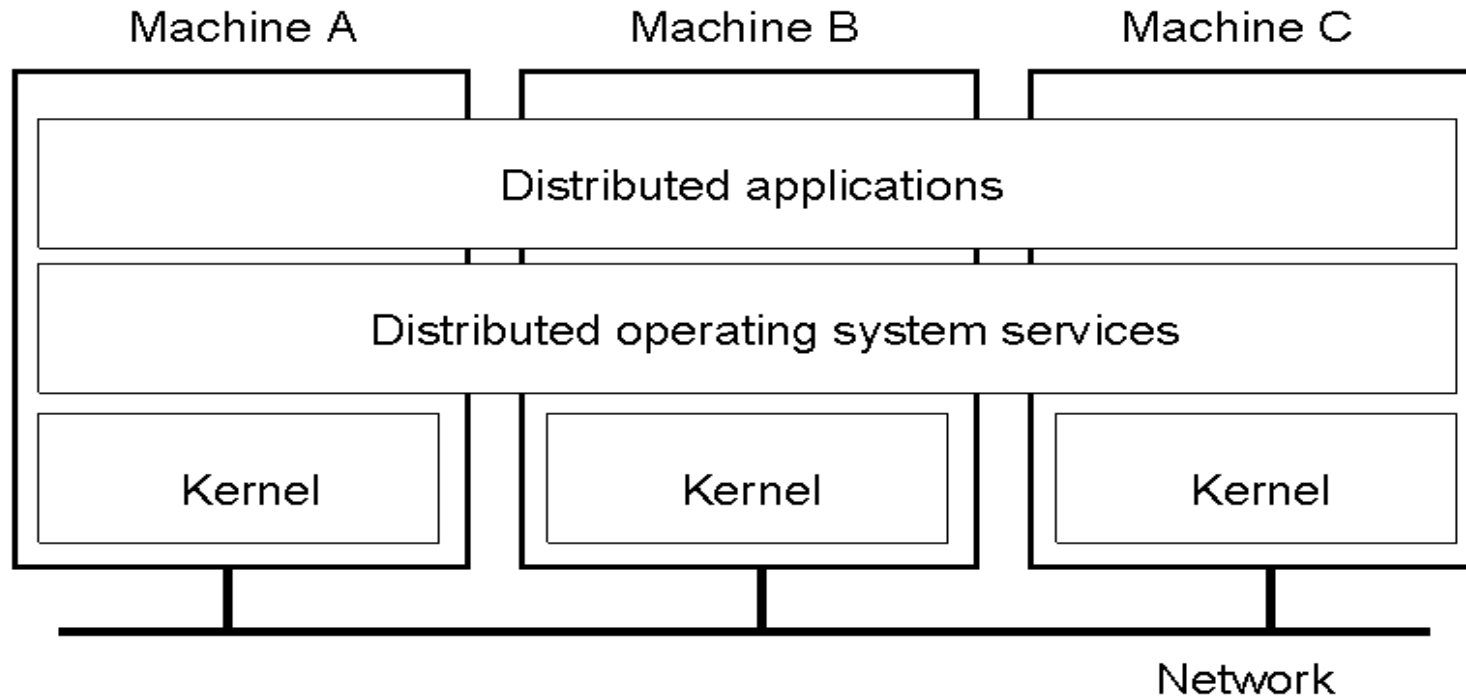
Distributed Operating System

- Manages resources in a distributed system
 - Seamlessly and transparently to the user
- Looks to the user like a centralized OS
 - But operates on multiple independent CPUs
- Provides transparency
 - Location, migration, concurrency, replication,...
- Presents users with a virtual uniprocessor.

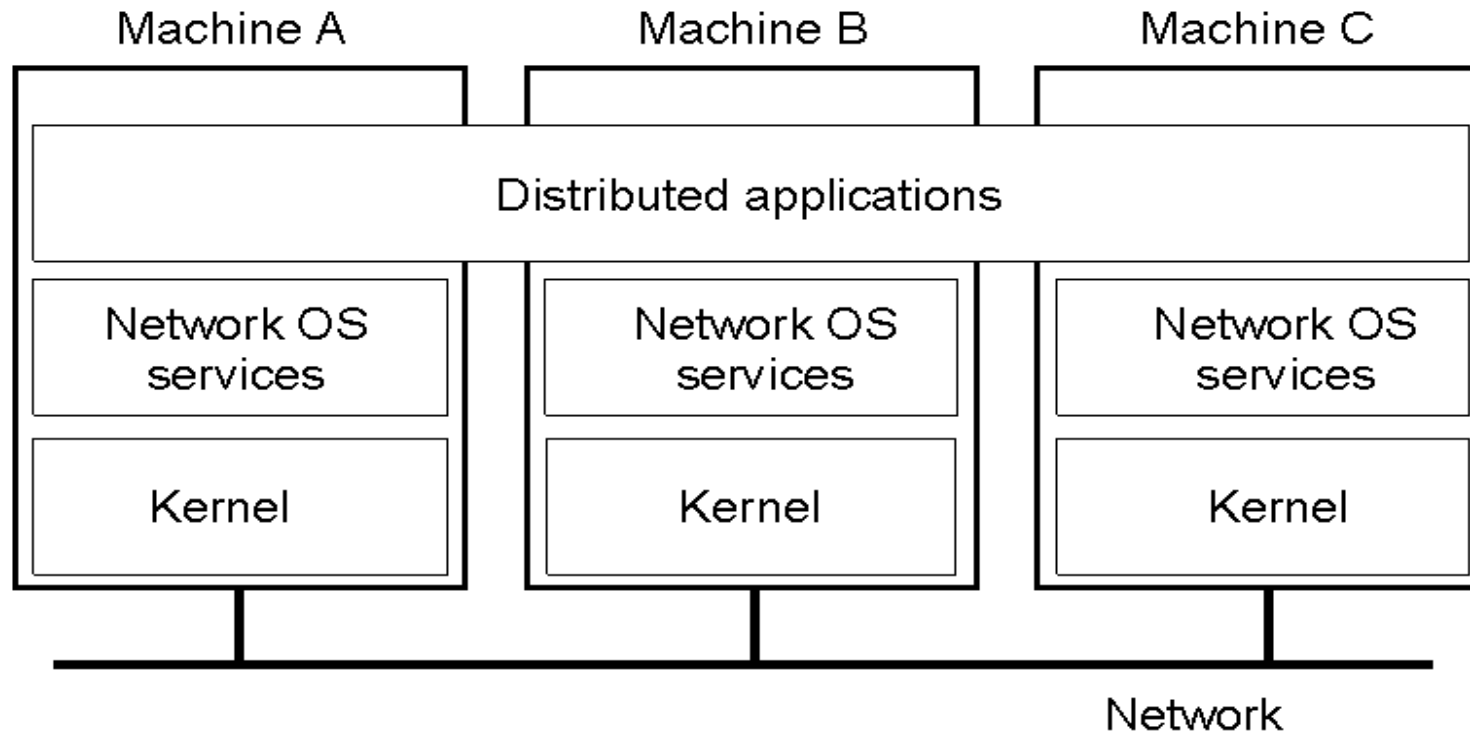
Types of Distributed OSs

System	Description	Main Goal
DOS	Tightly-coupled operating system for multi-processors and homogeneous multi-computers	Hide and manage hardware resources
NOS	Loosely-coupled operating system for heterogeneous multicomputers (LAN and WAN)	Offer local services to remote clients
Middleware	Additional layer atop of NOS implementing general-purpose services	Provide distribution transparency

Multicomputer Operating Systems

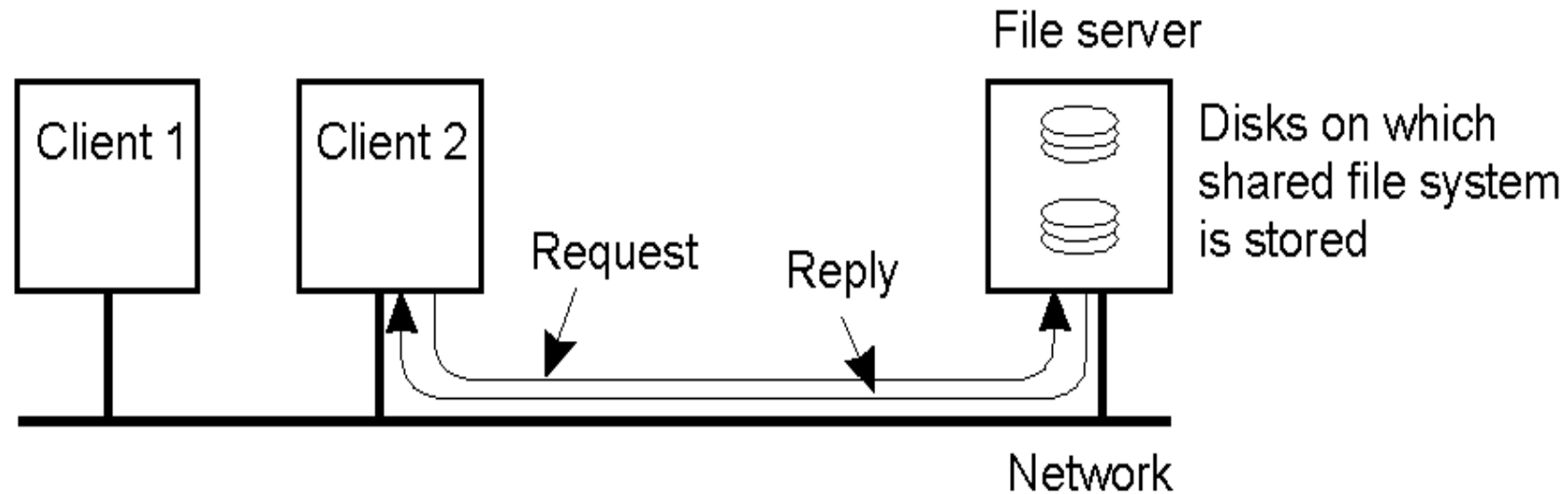


Network Operating System (I)



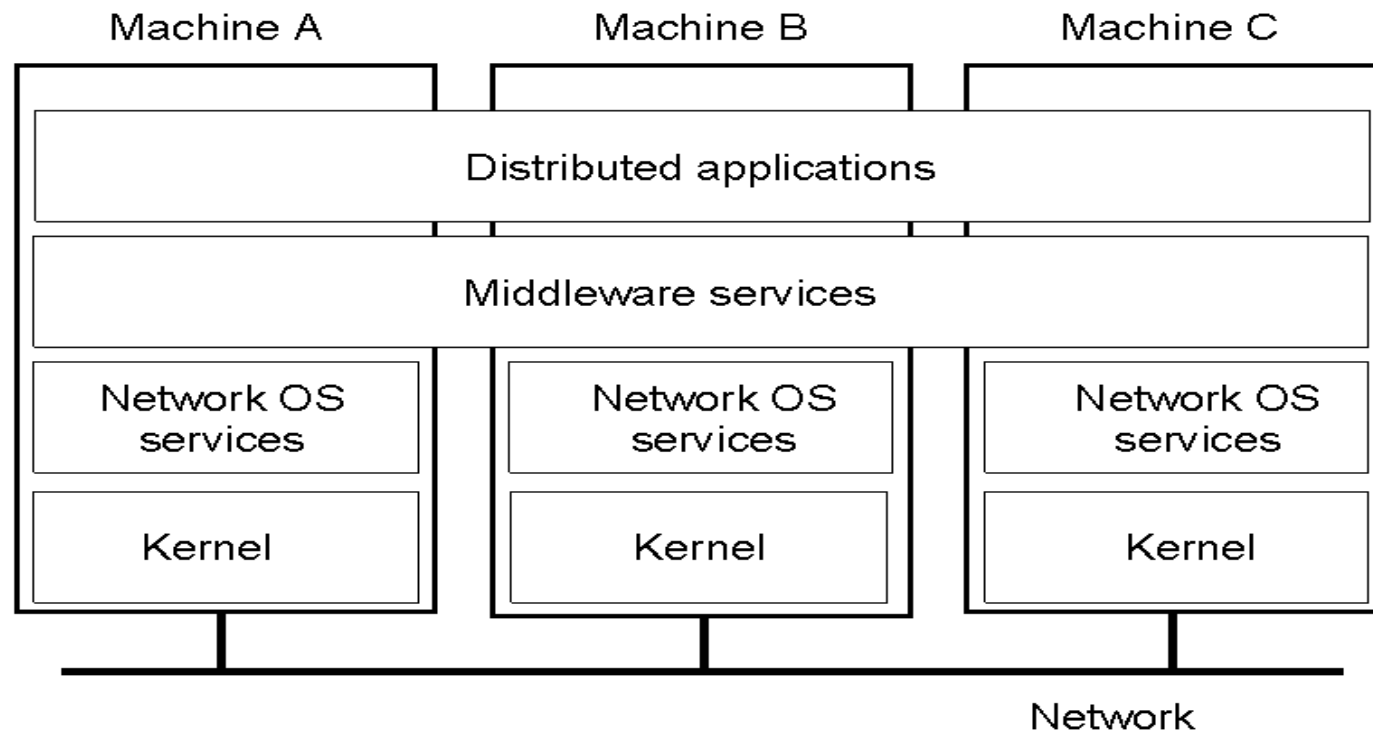
Network Operating System (2)

- Employs
 - Minim
 - Additic



Middleware-based systems

- General structure of a distributed system as middleware



Comparison between Systems

Item	Distributed OS		Network OS	Middleware-based OS
	Multiproc.	Multicomp.		
Degree of transparency	Very High	High	Low	High
Same OS on all nodes	Yes	Yes	No	No
Number of copies of OS	1	N	N	N
Basis for communication	Shared memory	Messages	Files	Model specific
Resource management	Global, central	Global, distributed	Per node	Per node
Scalability	No	Moderately	Yes	Varies
Openness	Closed	Closed	Open	Open